

Propuesta Técnica de Migración de Servicios entre Redes del LIS

Introducción

El Laboratorio Integrado de Sistemas (LIS) ha estado actualizando su infraestructura de red hacia el bloque institucional de la Universidad de Antioquia. Actualmente, tenemos servicios corriendo en la subred antigua *192.168.x.x* que deben migrarse al rango *10.18.30.x*.

Para este informe tomo como base un caso típico del laboratorio: una aplicación web dockerizada que corre en un servidor Linux. La arquitectura consta de un backend, una base de datos y Nginx actuando como proxy inverso. Cambiar la dirección IP de un servidor no es solo editar un archivo de red; implica revisar desde cómo resuelven las peticiones los clientes hasta las variables de entorno donde la IP quedó harcodeada. A continuación, detallo el diagnóstico, el flujo de migración, el plan de contingencia y la validación para hacer este cambio de forma limpia y controlada.

1. Diagnóstico del entorno actual

Antes de tocar cualquier configuración o bajar la interfaz de red, hay que levantar un mapa claro de cómo está operando la máquina y qué dependencias tiene. Si pasamos algo por alto en esta fase, el servicio va a fallar apenas hagamos el cambio de IP.

Red del host y rutas

Lo primero es validar cómo está respondiendo la interfaz actualmente. Con `ip addr show` identificamos la interfaz activa (por ejemplo, `eth0` o `ens33`), la IP *192.168.x.x* y la máscara. Luego revisamos la tabla de enrutamiento mediante `ip route show` para confirmar la IP del gateway y lanzamos un `ping -c 4 <gateway>` para asegurar la estabilidad de la red actual.

Resolución DNS y puertos expuestos

Necesitamos saber a qué DNS está apuntando el servidor (`cat /etc/resolv.conf`) y verificar qué registros externos apunta la aplicación. Si los usuarios entran mediante

un dominio (ej. servicio.lis.udea.edu.co), hacemos un `dig` o `nslookup` para registrar la dirección IP a la que apunta hoy.

A nivel de puertos, ejecutamos `ss -tulnp` para ver qué servicios están escuchando sockets en el host. Si la app corre sobre Docker, mapeamos los contenedores activos con `docker ps` y revisamos la asignación de puertos expuestos con `docker port <contenedor_id>`.

Búsqueda de IPs "quemadas" (Hardcoded)

Este es el punto más crítico. Usualmente quedan referencias a la red `192.168.x.x` guardadas en archivos de configuración, scripts de despliegue o código fuente. Para encontrarlas todas, lanzamos una búsqueda recursiva sobre el directorio del proyecto y las configuraciones del sistema:

```
grep -rn "192.168." /etc/nginx/ /opt/app/ /etc/environment
```

Hay que poner especial atención a:

- El archivo `docker-compose.yml` y los archivos `.env` (cadenas de conexión a la BD, variables como `REACT_APP_API_URL` o `ALLOWED_HOSTS`).
- La directiva `proxy_pass` en `/etc/nginx/sites-available/` si apunta a la IP local en vez de al nombre del contenedor o a `127.0.0.1`.
- Reglas locales de firewall (`sudo ufw status verbose` o `sudo iptables -L -n`).
- Certificados SSL/TLS: Si el certificado utiliza Subject Alternative Names (SAN) basados en la IP antigua en lugar del nombre de dominio, al cambiar la IP el navegador arrojará advertencias de seguridad.

2. Propuesta de migración

Una vez mapeado el sistema, se ejecutan las modificaciones requeridas dividiendo los cambios entre la capa de infraestructura y la capa de la aplicación.

Cambios a nivel de Red e Infraestructura

1. **Interfaz y Enrutamiento:** Se reconfigura el direccionamiento de la máquina virtual/servidor asignando la nueva IP dentro del bloque *10.18.30.x*, ajustando la máscara e indicando el nuevo gateway predeterminado de la UdeA.
2. **Ajuste de DNS y TTL:** En el servidor DNS de la universidad se debe actualizar el registro A del dominio para que apunte a la IP *10.18.30.x*. Se recomienda que entre 24 y 48 horas antes del procedimiento, se debe bajar el TTL (Time To Live) del registro a un valor bajo (ej. 300 segundos) para evitar que los clientes guarden la IP vieja en caché por mucho tiempo.
3. **Firewall y NAT:** Modificar las reglas en UFW/iptables sustituyendo el tráfico permitido a la subred *192.168.x.0/24* por *10.18.30.0/24*. Si hay un firewall perimetral o un router haciendo NAT hacia el servidor, se debe remapear la IP pública/interna a la nueva dirección.

Cambios en la Aplicación y Contenedores

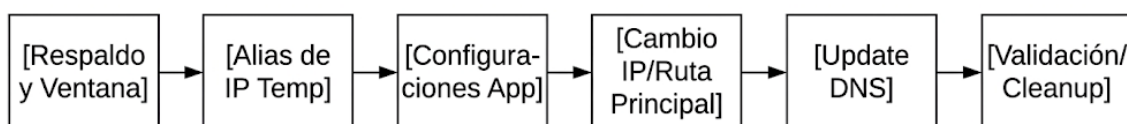
A nivel del proxy inverso (Nginx), si corregimos directivas como `proxy_pass` o `listen` que antes apuntaban explícitamente a la IP *192.168.x.x*, lo ideal no es solo cambiar el número por la nueva IP. La mejor práctica aquí es dejar de usar direcciones estáticas y pasar a referenciar directamente el nombre del servicio en la red bridge de Docker (por ejemplo: `proxy_pass http://backend_container:8080;`). Esto nos evita dolores de cabeza en futuras migraciones.

También se deben actualizar los archivos `.env` con las nuevas variables. Si el backend maneja restricciones de origen mediante CORS, añadimos la nueva IP (o mejor aún, modificamos la regla para que valide por el nombre de dominio institucional).

Por último, hay que prestar atención a los contenedores levantados con `network_mode: "host"`, ya que dependen por completo de la IP de la máquina física. Lo ideal en estos casos es aislarlos creando redes personalizadas con `docker network create`, rompiendo la dependencia directa con la red del host.

3. Plan de ejecución y reversión

Para minimizar la interrupción del servicio, los pasos deben seguir un orden riguroso.



Proceso de Ejecución

- **Fase Previa:** Notificar la ventana de mantenimiento, realizar respaldo completo de las bases de datos (dumps de Postgres/MySQL), copiar los archivos `.env`, configuraciones de Nginx y exportar las reglas de UFW actualizadas. Reducir el TTL del DNS.
- **Paso Intermedio (Conservar acceso):** Configurar la nueva IP `10.18.30.x` como una dirección secundaria (alias) en la interfaz de red. Esto evita perder la sesión SSH si estamos trabajando remotamente.
- **Despliegue de Cambios:** Actualizar archivos `.env`, la configuración de Nginx y el firewall. Reiniciar los contenedores con `docker compose down && docker compose up -d`.
- **Corte y Conmutación:** Cambiar el gateway por defecto a la red `10.18.30.x` y actualizar el registro DNS.
- **Criterio de Éxito:** Se considera exitosa la migración cuando la API/Web responde con HTTP 200 desde la nueva IP y el dominio, las consultas a la base de datos no presentan timeouts y `docker logs` no muestra errores de conexión.

Plan de Rollback (Reversión)

Si pasan 30 minutos desde la ventana y el servicio no logra conectarse a las dependencias o la red `10.18.30.x` presenta un fallo irreparable de enrutamiento:

1. Restaurar las variables de red del sistema para volver a usar la interfaz en `192.168.x.x`.
2. Revertir el registro DNS apuntando a la IP anterior (por eso bajamos el TTL antes).
3. Restaurar las versiones de respaldo de `.env`, `docker-compose.yml` y Nginx.
4. Reiniciar los servicios de Docker y el daemon de red (`systemctl restart networking / NetworkManager`).

La eliminación definitiva del alias de la IP `192.168.x.x` en la interfaz solo se realiza tras 24 horas de operación estable en la red nueva.

4. Pruebas de validación posterior

Terminadas las configuraciones, el protocolo de verificación se divide en tres niveles críticos para asegurar que no queden cabos sueltos.

- **Conectividad de bajo nivel:** Lo primero es corroborar que la interfaz levantó la nueva IP asignada usando `ip addr show`. Acto seguido, verificamos que el gateway institucional responda correctamente (`ping -c 4 10.18.30.1`) y ejecutamos un `traceroute 10.18.30.1` para confirmar que las rutas de salida no estén buscando interfaces obsoletas o segmentos de la red vieja.
- **Resolución de nombres y sockets:** Para validar el comportamiento de los puertos fuera del servidor, lanzamos un escaneo rápido desde otra máquina de la red:

```
> nc -zv 10.18.30.x 80 443
> dig @8.8.8.8 servicio.lis.udea.edu.co
```

Con `ss -tulnp | grep -E ':80|:443'` corroboramos de forma interna que Nginx efectivamente esté escuchando las peticiones en los sockets correctos.

- **Pruebas de la aplicación:** En la capa superior, validamos la salud del servicio web solicitando los encabezados directamente con `curl -I [https://servicio.lis.udea.edu.co/health](https://servicio.lis.udea.edu.co/health)`. Mientras hacemos estas peticiones de prueba, dejamos corriendo un `docker logs --tail 50 -f <contenedor_backend>` para asegurarnos de que la conexión interna hacia la base de datos esté limpia y no arroje excepciones de timeout.

5. Riesgos y puntos críticos

Durante este tipo de migraciones en entornos universitarios suelen presentarse imprevistos particulares:

- **El comportamiento del caché DNS:** Este es el problema clásico en la UdeA. Aunque cambiemos el registro en el DNS central, muchos routers intermedios o los mismos navegadores de los estudiantes seguirán intentando resolver hacia la `192.168.x.x` hasta que expire el TTL anterior. Por eso es vital no dar de baja la IP vieja inmediatamente y dejarla escuchando tráfico un par de horas más.
- **Cálculo erróneo de la máscara de subred:** Si por error configuramos una máscara `/24` en un segmento donde la infraestructura institucional requiere

una /23 o /22, el servidor quedará aislado y no podrá comunicarse con dependencias ubicadas en subredes hermanas de la universidad.

- **Aislamiento de bases de datos externas:** Si la base de datos corre en otro servidor del LIS y este no se migra al mismo tiempo, las reglas de firewall perimetrales de esa máquina van a rechazar las conexiones que vengan desde el nuevo rango *10.18.30.x*, tirando abajo la aplicación.
- **Bindeos explícitos en Docker:** Es muy común encontrar archivos `docker-compose.yml` con puertos declarados como *192.168.x.x:8080:8080*. Si la interfaz del host ya no tiene esa dirección asignada, los contenedores simplemente van a fallar al iniciar. La corrección inmediata es usar *0.0.0.0:8080* o declarar únicamente el puerto mapeado (*8080:8080*).

Referencias

Cloudflare. (s.f.). What is a subnet? Cloudflare Learning. Recuperado de <https://www.cloudflare.com/learning/network-layer/what-is-a-subnet/>

Cloudflare Developers. (s.f.). How Cloudflare DNS works. Cloudflare Docs. Recuperado de <https://developers.cloudflare.com/fundamentals/concepts/how-cloudflare-works/>

DigitalOcean. (s.f.). UFW essentials: Common firewall rules and commands for Linux security. DigitalOcean Community Tutorials. Recuperado de <https://www.digitalocean.com/community/tutorials/ufr-essentials-common-firewall-rules-and-commands>

Docker. (s.f.). Networking overview. Docker Docs. Recuperado de <https://docs.docker.com/engine/network/>

Fortinet. (s.f.). What is NAT (Network Address Translation)? Fortinet Cyberglossary. Recuperado de <https://www.fortinet.com/resources/cyberglossary/network-address-translation>

Gcore. (s.f.). What is DNS TTL? How it works, values, and best practices. Gcore Learning. Recuperado de <https://gcore.com/learning/what-is-dns-ttl>

MDN Web Docs. (s.f.). Cross-Origin Resource Sharing (CORS). Mozilla. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>

NGINX. (s.f.). NGINX reverse proxy. NGINX Documentation. Recuperado de <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>

Ubuntu. (s.f.). Firewall. Ubuntu Server Documentation. Recuperado de <https://ubuntu.com/server/docs/how-to/security/firewalls/>

Ubuntu. (s.f.). UFW. Ubuntu Community Help Wiki. Recuperado de <https://help.ubuntu.com/community/UFW>